SMART HAS

Consolidação: PL/SQL Avançado e DevOps

Packages · Triggers · Dynamic SQL · CI/CD (GitHub Actions)

INTEGRANTE



Leonardo Marsiarelli
RM: 555366
Turma: 3SIOA

CONTEXTUALIZAÇÃO

O Smart HAS chega à sua maturidade

1. **O que já existe**
   App mobile (React Native), backend (Spring Boot + JWT + AI Logistics Extension), dashboard (Angular) e a camada Oracle (package, triggers).

2. **Objetivo desta etapa**
   Profissionalizar a entrega final: PL/SQL avançado (packages, triggers, Dynamic SQL) + pipeline de CI/CD que builda, testa e publica automaticamente.

3. **Resultado**
   Produto tecnicamente robusto, seguro, automatizado — a integração Oracle é validada a cada push, não só revisada manualmente.

PARTE 1 — PL/SQL

Package SMARTHAS_PKG: organização e reuso

Function

FN_CLASSIFY_BP

Classifica a pressão arterial

Function

FN_CALC_RISK_SCORE

Indicador de risco (0-100)

Function

FN_GET_USER_SUMMARY

Resumo formatado (CURSOR)

Procedure

PRC_REGISTER_CRITICAL_ALERT

Registra alerta crítico

Procedure

PRC_USER_DELIVERY_REPORT

Relatório de entregas (CURSOR+LOOP)

Procedure

PRC_SEARCH_READINGS

Busca flexível — Dynamic SQL

PARTE 1 — PL/SQL

Triggers: auditoria automática

Nenhuma camada da aplicação chama explicitamente a auditoria — o rastro é gerado pelo próprio banco.

TRG_SENSOR_ALERT_AUDIT

AFTER INSERT ON sensor_alert

Grava em AUDIT_LOG todo alerta criado — "registrar logs de alertas"

TRG_DELIVERY_STATUS_AUDIT

AFTER UPDATE OF status ON medication_delivery_request

Grava o antes/depois de cada mudança de status — "atualizar status automaticamente"

PARTE 1 — PL/SQL

Dynamic SQL: flexibilidade em tempo real

PRC_SEARCH_READINGS: filtros todos opcionais + ordenação escolhida em tempo de execução.

```
v_order_col := CASE UPPER(p_order_by)
  WHEN 'RISK_DESC' THEN 'smarthas_pkg.fn_calc_risk_score(...) DESC'
  ELSE 'r.measured_at DESC' END;

v_sql := 'SELECT ... WHERE 1=1';
IF p_user_id IS NOT NULL THEN v_sql := v_sql || ' AND r.user_id = :p_user_id'; END IF;
OPEN p_cursor FOR v_sql USING <binds conforme os filtros ativos>;
```

- WHERE montado por concatenação — só entram os filtros informados
- ORDER BY resolvido contra whitelist — bind variable não parametriza coluna
- Exposta via GET /api/oracle/readings/search (filtros + orderBy)

PARTE 2 — DEVOPS

Pipeline CI/CD: 3 jobs encadeados

1

build-test `mvn test + package`

Compila, roda os testes e publica JAR + relatório de testes como artifacts

2

oracle-integration `Oracle XE real (service)`

Implanta schema + seed + PACKAGE + TRIGGERS e roda os exemplos de uso como teste de integração

3

deploy `Docker + GHCR`

Builda a imagem (Dockerfile multi-stage) e publica no GitHub Container Registry por commit

PARTE 2 — DEVOPS

Rodando de verdade no GitHub Actions

Não é só YAML revisado — o pipeline foi disparado e executado de ponta a ponta.

- ✓ build-test: testes automatizados executados e JAR publicado como artifact
- ✓ oracle-integration: Oracle XE real subido como serviço do job
- ✓ Schema + seed + SMARTHAS_PKG + triggers implantados automaticamente
- ✓ 05_usage_examples.sql rodou como teste de integração (falha em qualquer erro não tratado)
- ✓ deploy: imagem Docker publicada no GitHub Container Registry

ARQUITETURA CONSOLIDADA

Do código ao deploy, de ponta a ponta

Mobile + Dashboard
React Native / Angular

Backend
Spring Boot (REST)

Oracle
SMARTHAS_PKG + Triggers

↓ *git push*

build-test | **oracle-integration** | **deploy (GHCR)**

Pipeline CI/CD (GitHub Actions)

DEMONSTRAÇÃO & ENTREGA

Código, credenciais e vídeo

REPOSITÓRIO GITHUB

github.com/LeoMarsiarelli/Smart-HAS

/backend (+Dockerfile) · /database · /.github/workflows · /docs

VÍDEO DE DEMONSTRAÇÃO

[Inserir link do YouTube — não listado]

Demonstração prática do app + pipeline CI/CD em funcionamento

CREDENCIAIS DE DEMONSTRAÇÃO

ADMIN

admin@smarthas.com
senha: admin123

PACIENTE

maria@smarthas.com
senha: paciente123

Conclusão

O Smart HAS fecha esta etapa como um produto tecnicamente robusto: inteligência de negócio organizada em um package PL/SQL reutilizável, automação via triggers, consultas flexíveis via Dynamic SQL, e um pipeline de CI/CD que builda, testa contra um Oracle real e publica a aplicação automaticamente.

1. **Deploy em nuvem (ex.: Oracle Autonomous Database)**
2. **Observabilidade: métricas e alertas do pipeline**
3. **Motor de regras evoluindo para Machine Learning**

